```
1 0 1 1 0 1 0 0 1 1 1 0 1 0 1

0 1 1 0 0 1 1

x^8+x^7+x^6+x^4+1

H(X) = -sum p log2 p
```

INFORMATION THEORY & COMPRESSION ALGORITHMS

APPENDIX B

MATLAB Implementations

Entropy • Shannon–Fano • Huffman • Arithmetic • LZW • Hamming & SECDED • Repetition • BCH(15,7) • Markov • Capacity • Anomaly detection • Final project • Tested against the Python library

Mahmoud Abdelaziz
Military Technical College, Cairo

Companion to the first edition • 2026

Appendix B — MATLAB Implementations

B.1 Overview

This appendix gives a MATLAB version of every laboratory of Chapter 15 and of the final project. The algorithms, conventions and examples are the same as in the Python code, so all worked examples of the book can be reproduced in MATLAB. Copy all `.m` files (and the four `data_*.txt` files and `pyref_it.mat`) into one folder and run the lab scripts.

Functions	Purpose	Lab script
`entropy_bits, info_measures, kl_bits`	entropy, joint/conditional entropy, MI, KL	`lab1_entropy`
`shannon_fano_code, huffman_code, prefix_encode, prefix_decode`	prefix codes	`lab2_lab3_codes`
`arith_interval, arith_encode, arith_decode`	exact intervals; 32-bit integer coder	`lab4_arithmetic`
`lzw_encode, lzw_decode, lzw_bits`	LZW with dynamic widths	`lab5_lzw`
`compare_coders`	fair comparison with headers	`lab6_compare`
`hamming74_encode, hamming74_decode, secded84_decode`	Hamming and SECDED	`lab7_hamming`
`rep_decode, gf16_tables, bch1517_encode, bch1517_decode`	(5,1) repetition, BCH(15,7)	`lab8_multibit`
`markov_stationary, markov_entropy_rate, markov_estimate, markov_simulate`	Markov sources	`lab9_markov`
`channel_mi, blahut_arimoto`	capacity	`lab10_channel`
—	anomaly detection	`lab11_anomaly`
—	final project	`final_project_lab`
—	tests against Python	`test_itlab_matlab`

Requirements and testing

MATLAB R2016b or newer (implicit expansion, `containers.Map`, `repelem`); no toolbox is needed. All outputs printed here are real outputs from GNU Octave 8.4, which runs the same code. Deterministic results (codes, intervals, codewords, syndromes, capacities) are

identical to Python. Random experiments (Labs 9 and 11, the project's BSC) use MATLAB's own random generator, so their numbers differ slightly from the Python runs but agree statistically.

! MATLAB conventions

- Arrays start at 1: bit position j of the book's Hamming layout is `r(j)`, but BCH positions and LZW codes are 0-based numbers, as in the book; the functions handle the $+1$.
- Text is a `char` row vector; bit strings are `char` rows of `'0'` and `'1'`; bit vectors are `double` 0/1.
- The integer arithmetic coder uses doubles; keep the total frequency below 2^{16} so that all products stay exact ($< 2^{53}$).

B.2 Lab 1 — Entropy calculator

entropy_bits.m

```
function H = entropy_bits(p)
%ENTROPY_BITS  Shannon entropy H = -sum p log2 p (bits); zero probabilities skipped.
p = p(:);
if abs(sum(p) - 1) > 1e-9, error('probabilities must sum to 1'); end
p = p(p > 0);
H = -sum(p .* log2(p));
end
```

info_measures.m

```
function r = info_measures(P)
%INFO_MEASURES  Joint table P(x,y) (rows = X, columns = Y) -> struct with
%   HX, HY, HXY, HY_given_X, HX_given_Y, I (all in bits).
px = sum(P, 2);  py = sum(P, 1);
r.HX  = entropy_bits(px);
r.HY  = entropy_bits(py);
r.HXY = entropy_bits(P(:));
r.HY_given_X = r.HXY - r.HX;
r.HX_given_Y = r.HXY - r.HY;
r.I = r.HX + r.HY - r.HXY;
end
```

kl_bits.m

```
function D = kl_bits(p, q)
%KL_BITS  Kullback-Leibler divergence D(p||q) in bits (Inf if q=0 where p>0).
p = p(:); q = q(:);
k = p > 0;
if any(q(k) == 0), D = Inf; return; end
D = sum(p(k) .* log2(p(k) ./ q(k)));
end
```

lab1_entropy.m

```
%LAB1_ENTROPY  Entropy calculator (MATLAB version of entropy.py).
text = 'ABRACADABRA';
[sym, ~, idx] = unique(text);
p = accumarray(idx(:), 1)' / numel(text);
fprintf('symbols       : %s\n', sym);
fprintf('probabilities : %s\n', mat2str(round(p*1e4)/1e4));
fprintf('H(X)          : %.4f bits/symbol\n', entropy_bits(p));
fprintf('max possible  : %.4f bits/symbol\n', log2(numel(sym)));
fprintf('H2(0.1)       : %.4f bits\n', entropy_bits([0.1 0.9]));
r = info_measures([0.25 0.25; 0 0.5]);
fprintf('H(X,Y) = %.4f, H(Y|X) = %.4f, I(X;Y) = %.4f\n', r.HXY, r.HY_given_X, r.I);
fprintf('min-entropy of [0.6 0.2 0.1 0.1]: %.4f bits\n', -log2(0.6));
```

OUTPUT (real run)

```
symbols       : ABCDR
probabilities : [0.4545 0.1818 0.0909 0.0909 0.1818]
H(X)          : 2.0404 bits/symbol
max possible  : 2.3219 bits/symbol
H2(0.1)       : 0.4690 bits
H(X,Y) = 1.5000, H(Y|X) = 0.5000, I(X;Y) = 0.3113
min-entropy of [0.6 0.2 0.1 0.1]: 0.7370 bits
```

B.3 Labs 2 and 3 — Shannon–Fano and Huffman

shannon_fano_code.m

```
function codes = shannon_fano_code(sym, p)
%SHANNON_FANO_CODE  Shannon-Fano codes. sym: char array, p: probabilities.
%   Returns a cell array of codeword strings in the order of sym.
%   Sort by decreasing p (ties by symbol); split where |P_left - P_right|
%   is smallest (first such split); left part gets '0', right part '1'.
[~, order] = sortrows([-p(:), double(sym(:))]);
codes = repmat({''}, 1, numel(sym));
codes = split_group(order(:)', p, codes);
end

function codes = split_group(g, p, codes)
if numel(g) < 2, return; end
total = sum(p(g)); run = 0; best = inf; k = 1;
for j = 1:numel(g) - 1
    run = run + p(g(j));
    d = abs(2*run - total);
    if d < best - 1e-12, best = d; k = j; end
end
for j = 1:k,           codes{g(j)} = [codes{g(j)} '0']; end
for j = k+1:numel(g), codes{g(j)} = [codes{g(j)} '1']; end
codes = split_group(g(1:k), p, codes);
codes = split_group(g(k+1:end), p, codes);
end
```

huffman_code.m

```
function [codes, merges] = huffman_code(sym, p)
%HUFFMAN_CODE  Huffman codes with the book's conventions:
%   nodes are kept with an 'age'; the two smallest (ties: older first) are
%   merged, the first removed gets bit '0', the second bit '1'.
n = numel(sym);
codes = repmat({''}, 1, n);
if n == 1, codes{1} = '0'; merges = {}; return; end
[~, alpha] = sort(double(sym));                % leaves enter in symbol order
prob = p(alpha); members = num2cell(alpha); age = 1:n; nextage = n + 1;
merges = {};
while numel(prob) > 1
    [~, o] = sortrows([prob(:), age(:)]);         % smallest, then oldest
    a = o(1); b = o(2);
    for s = members{a}, codes{s} = ['0' codes{s}]; end
    for s = members{b}, codes{s} = ['1' codes{s}]; end
    merges(end+1, :) = {sym(members{a}), prob(a), sym(members{b}), prob(b)}; %#ok<AGROW>
    newp = prob(a) + prob(b); newm = [members{a} members{b}];
    keep = setdiff(1:numel(prob), [a b]);
    prob = [prob(keep) newp]; members = [members(keep) {newm}];
    age = [age(keep) nextage]; nextage = nextage + 1;
end
end
```

prefix_encode.m

```
function bits = prefix_encode(msg, sym, codes)
%PREFIX_ENCODE  Concatenate codewords of the symbols in msg.
bits = '';
for c = msg
    bits = [bits codes{sym == c}]; %#ok<AGROW>
end
end
```

prefix_decode.m

```
function msg = prefix_decode(bits, sym, codes)
%PREFIX_DECODE  Decode a prefix-free code by reading bits until a match.
msg = ''; cur = '';
for b = bits
    cur = [cur b]; %#ok<AGROW>
    k = find(strcmp(codes, cur), 1);
    if ~isempty(k)
        msg = [msg sym(k)]; %#ok<AGROW>
        cur = '';
    end
end
if ~isempty(cur), error('bitstream ended inside a codeword'); end
end
```

lab2_lab3_codes.m

```
%LAB2_LAB3_CODES  Shannon-Fano and Huffman (MATLAB versions of Labs 2 and 3).
sym = 'ABCDEF';  p = [0.30 0.25 0.15 0.12 0.10 0.08];
H = entropy_bits(p);
sf = shannon_fano_code(sym, p);
[hf, merges] = huffman_code(sym, p);
for k = 1:numel(sym)
    fprintf('%s  p = %.2f   Shannon-Fano %-4s  Huffman %-4s\n', sym(k), p(k), sf{k}, hf{k})
    ;
end
Lsf = sum(p .* cellfun(@numel, sf));  Lhf = sum(p .* cellfun(@numel, hf));
fprintf('H = %.4f  L(SF) = %.4f  L(Huffman) = %.4f  efficiency %.2f%%\n', H, Lsf, Lhf, 100*
    H/Lhf);
for k = 1:size(merges, 1)
    fprintf('merge %d: %s(%.2f) + %s(%.2f)\n', k, merges{k,1}, merges{k,2}, merges{k,3},
    merges{k,4});
end
msg = 'ABACADEF';
b = prefix_encode(msg, sym, hf);
fprintf('Huffman: %s -> %s -> %s\n', msg, b, prefix_decode(b, sym, hf));
```

OUTPUT (real run)

```
A  p = 0.30   Shannon-Fano 00    Huffman 11
B  p = 0.25   Shannon-Fano 01    Huffman 01
C  p = 0.15   Shannon-Fano 100   Huffman 101
D  p = 0.12   Shannon-Fano 101   Huffman 100
E  p = 0.10   Shannon-Fano 110   Huffman 001
F  p = 0.08   Shannon-Fano 111   Huffman 000
H = 2.4224  L(SF) = 2.4500  L(Huffman) = 2.4500  efficiency 98.87%
merge 1: F(0.08) + E(0.10)
merge 2: D(0.12) + C(0.15)
merge 3: FE(0.18) + B(0.25)
merge 4: DC(0.27) + A(0.30)
merge 5: FEB(0.43) + DCA(0.57)
Huffman: ABACADEF -> 11011110111100001000 -> ABACADEF
```

The merge table and the codewords are exactly those of Worked Example 7.1.

B.4 Lab 4 — Arithmetic coding

arith_interval.m

```
function [low, high, tag] = arith_interval(msg, sym, p)
%ARITH_INTERVAL  Teaching version: interval of a short message and the
%   shortest binary tag inside it (double precision: use for < 40 symbols).
[sym, o] = sort(sym); p = p(o);
Fh = cumsum(p); Fl = Fh - p;
low = 0; high = 1;
for c = msg
    k = find(sym == c, 1);
    w = high - low;
```

```
    high = low + w * Fh(k);
    low  = low + w * Fl(k);
    fprintf('  after %s: [%.6f, %.6f)  width %.6f\n', c, low, high, high - low);
end
k = 1;
while true
    m = ceil(low * 2^k - 1e-12);
    if (m + 1) / 2^k <= high + 1e-12
        tag = dec2bin(m, k); return
    end
    k = k + 1;
end
end
```

arith_encode.m

```
function bits = arith_encode(msg, sym, counts)
%ARITH_ENCODE  Integer arithmetic coder (32-bit, E1/E2/E3 scaling).
%   msg: char row; sym: char row of symbols (any order); counts: integer
%   frequencies (total <= 2^16 so that all products stay exact in doubles).
%   Symbols are placed in the cumulative table in sorted order.
[sym, o] = sort(sym); counts = counts(o);
cumh = cumsum(counts); cuml = cumh - counts; total = cumh(end);
FULL = 2^32 - 1; HALF = 2^31; QUART = 2^30;
low = 0; high = FULL; pending = 0; bits = blanks(0);
for c = msg
    k = find(sym == c, 1);
    rng = high - low + 1;
    high = low + floor(rng * cumh(k) / total) - 1;
    low  = low + floor(rng * cuml(k) / total);
    while true
        if high < HALF
            [bits, pending] = emit(bits, '0', pending);
        elseif low >= HALF
            [bits, pending] = emit(bits, '1', pending);
            low = low - HALF; high = high - HALF;
        elseif low >= QUART && high < 3*QUART
            pending = pending + 1; low = low - QUART; high = high - QUART;
        else
            break
        end
        low = 2*low; high = 2*high + 1;
    end
end
pending = pending + 1;
if low < QUART, [bits, ~] = emit(bits, '0', pending);
else,           [bits, ~] = emit(bits, '1', pending); end
end

function [bits, pending] = emit(bits, b, pending)
bits = [bits b repmat(char('0' + ('1' - b)), 1, pending)];
pending = 0;
end
```

arith_decode.m

```
function msg = arith_decode(bits, sym, counts, n)
%ARITH_DECODE  Decoder for ARITH_ENCODE; n = number of symbols to decode.
[sym, o] = sort(sym); counts = counts(o);
cumh = cumsum(counts); cuml = cumh - counts; total = cumh(end);
FULL = 2^32 - 1; HALF = 2^31; QUART = 2^30;
stream = bits - '0'; pos = 0;
low = 0; high = FULL; code = 0;
for i = 1:32, [b, pos] = nextbit(stream, pos); code = 2*code + b; end
msg = blanks(n);
for i = 1:n
    rng = high - low + 1;
    target = floor(((code - low + 1) * total - 1) / rng);
    k = find(target >= cuml & target < cumh, 1);
    msg(i) = sym(k);
    high = low + floor(rng * cumh(k) / total) - 1;
    low  = low + floor(rng * cuml(k) / total);
    while true
        if high < HALF
            % nothing to subtract
        elseif low >= HALF
            low = low - HALF; high = high - HALF; code = code - HALF;
        elseif low >= QUART && high < 3*QUART
            low = low - QUART; high = high - QUART; code = code - QUART;
        else
            break
        end
        low = 2*low; high = 2*high + 1;
        [b, pos] = nextbit(stream, pos); code = 2*code + b;
    end
end
end

function [b, pos] = nextbit(stream, pos)
pos = pos + 1;
if pos <= numel(stream), b = stream(pos); else, b = 0; end
end
```

lab4_arithmetic.m

```
%LAB4_ARITHMETIC  Arithmetic coding (MATLAB version of arithmetic.py).
disp('Exact example: p(A)=1/2, p(B)=1/4, p(C)=1/4, message ABAC');
[lo, hi, tag] = arith_interval('ABAC', 'ABC', [0.5 0.25 0.25]);
fprintf('  tag bits: %s\n', tag);
text = 'TOBEORNOTTOBEORTOBEORNOT';
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)';
b = arith_encode(text, sym, counts);
fprintf('integer coder: %d bits; round trip OK: %d\n', numel(b), ...
        strcmp(arith_decode(b, sym, counts, numel(text)), text));
```

OUTPUT (real run)

```
Exact example: p(A)=1/2, p(B)=1/4, p(C)=1/4, message ABAC
  after A: [0.000000, 0.500000)  width 0.500000
  after B: [0.250000, 0.375000)  width 0.125000
  after A: [0.250000, 0.312500)  width 0.062500
  after C: [0.296875, 0.312500)  width 0.015625
  tag bits: 010011
integer coder: 60 bits; round trip OK: 1
```

B.5 Lab 5 — LZW

lzw_encode.m

```
function codes = lzw_encode(text, alphabet)
%LZW_ENCODE  LZW codes. alphabet = '' uses the 256 byte values (code = ASCII).
if nargin < 2 || isempty(alphabet), alphabet = char(0:255); end
d = containers.Map('KeyType', 'char', 'ValueType', 'double');
for i = 1:numel(alphabet), d(alphabet(i)) = i - 1; end
w = ''; codes = [];
for c = text
    if isKey(d, [w c])
        w = [w c];
    else
        codes(end+1) = d(w); %#ok<AGROW>
        d([w c]) = d.Count;      % next free code
        w = c;
    end
end
if ~isempty(w), codes(end+1) = d(w); end
end
```

lzw_decode.m

```
function text = lzw_decode(codes, alphabet)
%LZW_DECODE  Rebuilds the dictionary while decoding; handles the cScSc case.
if nargin < 2 || isempty(alphabet), alphabet = char(0:255); end
d = cell(1, numel(alphabet) + numel(codes));
for i = 1:numel(alphabet), d{i} = alphabet(i); end
n = numel(alphabet);                          % number of entries
w = d{codes(1) + 1}; text = w;
for k = codes(2:end)
    if k < n
        entry = d{k + 1};
    elseif k == n
        entry = [w w(1)];                     % special case
    else
        error('bad code %d', k);
    end
    n = n + 1; d{n} = [w entry(1)];
    text = [text entry]; %#ok<AGROW>
    w = entry;
```

```
end
end
```

lzw_bits.m

```
function total = lzw_bits(codes, n0)
%LZW_BITS  Bits with dynamic widths: code k (0-based) uses ceil(log2(n0+k)).
k = 0:numel(codes) - 1;
total = sum(max(1, ceil(log2(n0 + k))));
end
```

lab5_lzw.m

```
%LAB5_LZW  LZW compressor and decompressor (MATLAB version of lzw.py).
c = lzw_encode('ABABABA', 'AB');
fprintf('ABABABA -> %s -> %s, %d bits\n', mat2str(c), lzw_decode(c, 'AB'), lzw_bits(c, 2));
t = 'TOBEORNOTTOBEORTOBEORNOT';
c = lzw_encode(t, '');
fprintf('%s\n  codes: %s\n  round trip OK: %d, %d bits (original %d)\n', t, mat2str(c), ...
        strcmp(lzw_decode(c, ''), t), lzw_bits(c, 256), 8*numel(t));
```

OUTPUT (real run)

```
ABABABA -> [0 1 2 4] -> ABABABA, 8 bits
TOBEORNOTTOBEORTOBEORNOT
  codes: [84 79 66 69 79 82 78 79 84 256 258 260 265 259 261 263]
  round trip OK: 1, 143 bits (original 192)
```

B.6 Lab 6 — Compression comparison

compare_coders.m

```
function R = compare_coders(text)
%COMPARE_CODERS  Payload and header bits of the four coders, with
%   round-trip checks (header: 16 bits + 24 bits per symbol, as in Python).
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)'; p = counts / numel(text);
hdr = 16 + numel(sym) * 24;
sf = shannon_fano_code(sym, p);  b = prefix_encode(text, sym, sf);
assert(strcmp(prefix_decode(b, sym, sf), text)); R.sf = [numel(b) hdr];
hf = huffman_code(sym, p);       b = prefix_encode(text, sym, hf);
assert(strcmp(prefix_decode(b, sym, hf), text)); R.huffman = [numel(b) hdr];
b = arith_encode(text, sym, counts);
assert(strcmp(arith_decode(b, sym, counts, numel(text)), text)); R.arith = [numel(b) hdr];
c = lzw_encode(text, '');
assert(strcmp(lzw_decode(c, ''), text)); R.lzw = [lzw_bits(c, 256) 0];
R.H = entropy_bits(p); R.n = numel(text);
end
```

lab6_compare.m

```
%LAB6_COMPARE  Fair comparison on the four data sets of Chapter 10
%   (the text files were written by compare.py so both languages use the same data).
names = {'skewed', 'english', 'repetitive', 'uniform'};
for k = 1:numel(names)
    fid = fopen(['data_' names{k} '.txt']); text = fgetl(fid); fclose(fid);
    R = compare_coders(text);
    fprintf('%-10s n = %4d  H = %.3f  nH = %4.0f |', names{k}, R.n, R.H, R.n * R.H);
    fprintf(' SF %d+%d | Huffman %d+%d | Arith %d+%d | LZW %d\n', R.sf, R.huffman, R.arith,
     R.lzw(1));
end
```

OUTPUT (real run)

```
skewed     n = 1000  H = 1.641  nH = 1641 | SF 1680+136 | Huffman 1680+136 | Arith 1643+136 | LZW 2642
english    n = 1020  H = 4.122  nH = 4204 | SF 4266+640 | Huffman 4240+640 | Arith 4205+640 | LZW 4832
repetitive n = 1080  H = 2.419  nH = 2613 | SF 2640+160 | Huffman 2640+160 | Arith 2614+160 | LZW 1214
uniform    n = 1000  H = 3.994  nH = 3994 | SF 4033+400 | Huffman 4000+400 | Arith 3995+400 | LZW 5752
```

Every number is identical to the Python results of Chapter 10.

B.7 Lab 7 — Hamming and SECDED

hamming74_encode.m

```
function c = hamming74_encode(d)
%HAMMING74_ENCODE  d = [d1 d2 d3 d4] -> [p1 p2 d1 p4 d2 d3 d4] (positions 1..7).
p1 = xor(xor(d(1), d(2)), d(4));
p2 = xor(xor(d(1), d(3)), d(4));
p4 = xor(xor(d(2), d(3)), d(4));
c = double([p1 p2 d(1) p4 d(2) d(3) d(4)]);
end
```

hamming74_decode.m

```
function [d, r, s] = hamming74_decode(r)
%HAMMING74_DECODE  Single-error correction. s = syndrome = error position.
H = [1 0 1 0 1 0 1; 0 1 1 0 0 1 1; 0 0 0 1 1 1 1];   % rows s1, s2, s4
sv = mod(H * r(:), 2);
s = sv(1) + 2*sv(2) + 4*sv(3);
if s > 0, r(s) = 1 - r(s); end
d = r([3 5 6 7]);
end
```

secded84_decode.m

```
function [d, status] = secded84_decode(r)
%SECDED84_DECODE  Extended Hamming (8,4): r = [7-bit Hamming word, p0].
[~, ~, s] = hamming74_decode(r(1:7));
overall = mod(sum(r), 2);
if s == 0 && overall == 0
```

```
    d = r([3 5 6 7]); status = 'no error';
elseif overall == 1 && s == 0
    d = r([3 5 6 7]); status = 'parity bit error';
elseif overall == 1
    d = hamming74_decode(r(1:7)); status = 'corrected single error';
else
    d = []; status = 'double error detected';
end
end
```

lab7_hamming.m

```
%LAB7_HAMMING  Hamming (7,4) and SECDED (MATLAB version of hamming.py).
d = [1 0 1 1];
c = hamming74_encode(d);
fprintf('data %s -> codeword %s\n', mat2str(d), mat2str(c));
r = c; r(5) = 1 - r(5);                          % error at position 5
[dd, fixed, s] = hamming74_decode(r);
fprintf('received %s syndrome %d -> corrected %s data %s\n', mat2str(r), s, mat2str(fixed),
     mat2str(dd));
r2 = c; r2([2 6]) = 1 - r2([2 6]);               % two errors
[dd2, ~, s2] = hamming74_decode(r2);
fprintf('two errors -> syndrome %d data %s (WRONG)\n', s2, mat2str(dd2));
c8 = [c mod(sum(c), 2)];  r8 = c8; r8([2 6]) = 1 - r8([2 6]);
[~, st] = secded84_decode(r8);
fprintf('SECDED with the same two errors -> %s\n', st);
```

OUTPUT (real run)

```
data [1 0 1 1] -> codeword [0 1 1 0 0 1 1]
received [0 1 1 0 1 1 1] syndrome 5 -> corrected [0 1 1 0 0 1 1] data [1 0 1 1]
two errors -> syndrome 4 data [1 0 0 1] (WRONG)
SECDED with the same two errors -> double error detected
```

B.8 Lab 8 — Repetition and BCH(15,7)

rep_decode.m

```
function [bits, votes] = rep_decode(r, n)
%REP_DECODE  Majority vote on blocks of n bits (n odd, default 5).
if nargin < 2, n = 5; end
blocks = reshape(r, n, []);
votes = sum(blocks, 1);
bits = double(votes > floor(n/2));
end
```

gf16_tables.m

```
function [EXP, LOG] = gf16_tables()
%GF16_TABLES  EXP(i+1) = alpha^i (i = 0..29) as integers 0..15, from x^4+x+1.
%   LOG(v+1) = i with alpha^i = v (v = 1..15).
```

```
EXP = zeros(1, 30); LOG = zeros(1, 16); v = 1;
for i = 0:14
    EXP(i+1) = v; LOG(v+1) = i;
    v = v * 2;
    if v >= 16, v = bitxor(v, 19); end          % 19 = 10011b
end
EXP(16:30) = EXP(1:15);
end
```

bch1517_encode.m

```
function c = bch1517_encode(m)
%BCH1517_ENCODE  Systematic BCH(15,7): c = [parity(8) message(7)],
%   coefficient order x^0..x^14, g(x) = 1 + x^4 + x^6 + x^7 + x^8.
g = [1 0 0 0 1 0 1 1 1];
r = [zeros(1, 8) m(:)'];                        % x^8 m(x)
for i = 15:-1:9                                 % long division
    if r(i)
        r(i-8:i) = xor(r(i-8:i), g);
    end
end
c = [double(r(1:8)) m(:)'];
end
```

bch1517_decode.m

```
function [m, c, pos] = bch1517_decode(r)
%BCH1517_DECODE  Peterson decoder, corrects up to 2 errors.
%   pos = 0-based error positions found ([] if none, NaN if failure).
[EXP, LOG] = gf16_tables();
mul = @(a, b) (a ~= 0 && b ~= 0) * EXP(mod(LOG(a+1) + LOG(b+1), 15) + 1);
dv  = @(a, b) (a ~= 0) * EXP(mod(LOG(a+1) - LOG(b+1), 15) + 1);
pw  = @(a, k) (a ~= 0) * EXP(mod(LOG(a+1) * k, 15) + 1);
s1 = 0; s3 = 0;
for i = 0:14
    if r(i+1)
        s1 = bitxor(s1, EXP(mod(i, 15) + 1));
        s3 = bitxor(s3, EXP(mod(3*i, 15) + 1));
    end
end
c = r; pos = [];
if s1 == 0 && s3 == 0
    m = r(9:15); return
end
if s1 ~= 0 && s3 == pw(s1, 3)
    pos = LOG(s1 + 1);
elseif s1 == 0
    m = r(9:15); pos = NaN; return
else
    sigma2 = dv(bitxor(s3, pw(s1, 3)), s1);
    for j = 0:14                                % Chien search, x = alpha^-j
        x = EXP(mod(15 - j, 15) + 1);
        if bitxor(bitxor(1, mul(s1, x)), mul(sigma2, mul(x, x))) == 0
            pos(end+1) = j; %#ok<AGROW>
```

```
        end
    end
    if numel(pos) ~= 2, m = r(9:15); pos = NaN; return; end
end
c(pos + 1) = 1 - c(pos + 1);
m = c(9:15);
end
```

lab8_multibit.m

```
%LAB8_MULTIBIT  (5,1) repetition and BCH(15,7) (MATLAB version of Lab 8).
msg = [1 0 1];
c = repelem(msg, 5);
r = c; r([2 4 7 10]) = 1 - r([2 4 7 10]);       % 2 errors in blocks 1 and 2
[dec, votes] = rep_decode(r);
fprintf('repetition: votes %s -> decoded %s\n', mat2str(votes), mat2str(dec));
r3 = c; r3(1:3) = 1 - r3(1:3);
fprintf('3 errors in one block -> %s (failure expected)\n', mat2str(rep_decode(r3)));
m = [1 0 1 1 0 0 1];
cw = bch1517_encode(m);
fprintf('BCH codeword %s\n', mat2str(cw));
rb = cw; rb([3 12]) = 1 - rb([3 12]);           % errors at 0-based positions 2 and 11
[mh, ~, pos] = bch1517_decode(rb);
fprintf('errors found at positions %s, message OK: %d\n', mat2str(pos), isequal(mh, m));
```

OUTPUT (real run)

```
repetition: votes [3 2 5] -> decoded [1 0 1]
3 errors in one block -> [0 0 1] (failure expected)
BCH codeword [0 1 0 0 0 0 1 1 1 0 1 1 0 0 1]
errors found at positions [2 11], message OK: 1
```

B.9 Lab 9 — Markov chains

markov_stationary.m

```
function pi_ = markov_stationary(P)
%MARKOV_STATIONARY  Solve pi*P = pi, sum(pi) = 1.
S = size(P, 1);
A = [P' - eye(S); ones(1, S)];
pi_ = (A \ [zeros(S, 1); 1])';
end
```

markov_entropy_rate.m

```
function H = markov_entropy_rate(P)
%MARKOV_ENTROPY_RATE  H = sum_i pi_i * H(row i), in bits/symbol.
pi_ = markov_stationary(P);
Hr = zeros(1, size(P, 1));
for i = 1:size(P, 1)
    row = P(i, P(i, :) > 0);
```

```
    Hr(i) = -sum(row .* log2(row));
end
H = sum(pi_ .* Hr);
end
```

markov_estimate.m

```
function P = markov_estimate(seq, S, alpha)
%MARKOV_ESTIMATE  ML transition matrix from a state sequence (states 1..S),
%   with optional add-alpha smoothing.
if nargin < 3, alpha = 0; end
N = alpha * ones(S);
for k = 2:numel(seq)
    N(seq(k-1), seq(k)) = N(seq(k-1), seq(k)) + 1;
end
P = N ./ sum(N, 2);
end
```

markov_simulate.m

```
function seq = markov_simulate(P, n, start)
%MARKOV_SIMULATE  n states of a chain (states 1..S); seed rng before calling.
seq = zeros(1, n); seq(1) = start; C = cumsum(P, 2);
for k = 2:n
    seq(k) = find(rand < C(seq(k-1), :), 1);
end
end
```

lab9_markov.m

```
%LAB9_MARKOV  Markov chains (MATLAB version of markov.py).
P = [0.9 0.1; 0.5 0.5];
pi_ = markov_stationary(P);
fprintf('stationary distribution: %s\n', mat2str(round(pi_*1e4)/1e4));
fprintf('entropy rate: %.4f bits/symbol\n', markov_entropy_rate(P));
fprintf('entropy of pi (memoryless view): %.4f\n', entropy_bits(pi_));
rng(1); seq = markov_simulate(P, 20000, 1);
fprintf('estimated P from 20000 samples: %s\n', mat2str(round(markov_estimate(seq, 2)*1e3)
    /1e3));
```

OUTPUT (real run)

```
stationary distribution: [0.8333 0.1667]
entropy rate: 0.5575 bits/symbol
entropy of pi (memoryless view): 0.6500
estimated P from 20000 samples: [0.898 0.102;0.495 0.505]
```

B.10 Lab 10 — Channel capacity

channel_mi.m

```
function I = channel_mi(px, W)
%CHANNEL_MI  I(X;Y) for input px (row) and channel W(x,y) = P(y|x).
py = px * W;
I = 0;
for x = 1:numel(px)
    for y = 1:size(W, 2)
        if px(x) > 0 && W(x, y) > 0
            I = I + px(x) * W(x, y) * log2(W(x, y) / py(y));
        end
    end
end
end
```

blahut_arimoto.m

```
function [C, px] = blahut_arimoto(W, iters)
%BLAHUT_ARIMOTO  Capacity of a DMC W(x,y) = P(y|x) and the best input.
if nargin < 2, iters = 2000; end
nx = size(W, 1); px = ones(1, nx) / nx;
for it = 1:iters
    py = px * W;
    D = zeros(1, nx);
    for x = 1:nx
        k = W(x, :) > 0;
        D(x) = sum(W(x, k) .* log2(W(x, k) ./ py(k)));
    end
    new = px .* 2.^D; new = new / sum(new);
    if max(abs(new - px)) < 1e-12, px = new; break; end
    px = new;
end
C = channel_mi(px, W);
end
```

lab10_channel.m

```
%LAB10_CHANNEL  Channel capacity (MATLAB version of channel.py).
h2 = @(p) -p.*log2(max(p, eps)) - (1-p).*log2(max(1-p, eps));
for p = [0 0.01 0.11 0.5]
    fprintf('BSC p = %.2f  C = %.4f bits/use\n', p, 1 - h2(p));
end
W = [0.9 0.1; 0.2 0.8];
[C, px] = blahut_arimoto(W);
fprintf('channel [0.9 0.1; 0.2 0.8]: C = %.4f, best P(X) = %s\n', C, mat2str(round(px*1e4)
    /1e4));
[C, px] = blahut_arimoto([1 0; 0.5 0.5]);
fprintf('Z-channel q = 0.5: C = %.4f, best P(X) = %s\n', C, mat2str(round(px*1e4)/1e4));
```

OUTPUT (real run)

```
BSC p = 0.00  C = 1.0000 bits/use
BSC p = 0.01  C = 0.9192 bits/use
BSC p = 0.11  C = 0.5001 bits/use
BSC p = 0.50  C = 0.0000 bits/use
channel [0.9 0.1; 0.2 0.8]: C = 0.3978, best P(X) = [0.5176 0.4824]
Z-channel q = 0.5: C = 0.3219, best P(X) = [0.6 0.4]
```

B.11 Lab 11 — Anomaly detection

lab11_anomaly.m

```
%LAB11_ANOMALY  KL- and log-loss-based anomaly detection (MATLAB version).
%   Actions 1 = login, 2 = read, 3 = write, 4 = logout.
Pn = [0.05 0.80 0.10 0.05; 0.02 0.60 0.25 0.13; 0.02 0.50 0.30 0.18; 0.90 0.04 0.03 0.03];
Pa = [0.60 0.10 0.05 0.25; 0.05 0.30 0.55 0.10; 0.05 0.20 0.65 0.10; 0.85 0.05 0.05 0.05];
W = 50; rng(7);
train = markov_simulate(Pn, 20000, 1);
Ph = markov_estimate(train, 4, 1);              % smoothed ML baseline
q = markov_stationary(Ph);
freqw = @(w) (accumarray(w(:), 1, [4 1])' + 0.5) / (numel(w) + 2);
klsc  = @(w) kl_bits(freqw(w), q);
llsc  = @(w) -mean(log2(Ph(sub2ind([4 4], w(1:end-1), w(2:end)))));
win = @(s) reshape(s(1:floor(numel(s)/W)*W), W, [])';
valid = win(markov_simulate(Pn, 200*W, 1));
tn = win(markov_simulate(Pn, 200*W, 1));
ta = win(markov_simulate(Pa, 200*W, 1));
scores = {klsc, llsc}; names = {'KL', 'log-loss'};
fprintf('baseline action frequencies: %s\n', mat2str(round(q*1e3)/1e3));
for s = 1:2
    f = scores{s};
    v = sort(arrayfun(@(i) f(valid(i, :)), 1:size(valid, 1)));
    thr = v(min(numel(v), floor(0.99*numel(v)) + 1));
    fp = mean(arrayfun(@(i) f(tn(i, :)), 1:size(tn, 1)) > thr);
    fn = mean(arrayfun(@(i) f(ta(i, :)), 1:size(ta, 1)) <= thr);
    fprintf('%-8s threshold %.3f  false positive rate %.3f  false negative rate %.3f\n',
    names{s}, thr, fp, fn);
end
```

OUTPUT (real run)

```
baseline action frequencies: [0.128 0.541 0.212 0.118]
KL       threshold 0.136  false positive rate 0.030  false negative rate 0.000
log-loss threshold 1.665  false positive rate 0.020  false negative rate 0.000
```

With MATLAB's random stream the 99th-percentile thresholds are a little lower than in the Python run, so 2–3% of normal test windows raise an alarm and no attack window is missed. This illustrates how thresholds chosen on 200 validation windows vary from run to run; use more validation data for stable thresholds.

B.12 Final project

final_project_lab.m

```
%FINAL_PROJECT_LAB  Information Theory and Compression Laboratory (MATLAB
%   version of project.py): statistics, four compressors, channel coding,
%   BSC errors, recovery.
text = repmat(['INFORMATION THEORY MEASURES SURPRISE. COMPRESSION REMOVES REDUNDANCY. ' ...
               'ERROR CORRECTION ADDS REDUNDANCY BACK, IN A CONTROLLED WAY. '], 1, 4);
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)'; p = counts / numel(text);
fprintf('STEP 1-3  %d symbols, %d distinct, H = %.4f bits/symbol\n', numel(text), numel(sym
    ), entropy_bits(p));
R = compare_coders(text);
fprintf('STEP 4-8  SF %d+%d, Huffman %d+%d, arithmetic %d+%d, LZW %d (raw %d)\n', ...
        R.sf, R.huffman, R.arith, R.lzw(1), 8*numel(text));
hf = huffman_code(sym, p);
src = prefix_encode(text, sym, hf) - '0';
pBSC = 0.01; rng(2026);
schemes = {'none', 'hamming74', 'rep5'};
for s = 1:3
    switch schemes{s}
        case 'none',      tx = src;
        case 'hamming74', pad = [src zeros(1, mod(-numel(src), 4))];
                          tx = cell2mat(arrayfun(@(i) hamming74_encode(pad(i:i+3)), 1:4:
    numel(pad), 'UniformOutput', false));
        case 'rep5',      tx = repelem(src, 5);
    end
    rx = double(xor(tx, rand(size(tx)) < pBSC));
    switch schemes{s}
        case 'none',      out = rx;
        case 'hamming74', out = cell2mat(arrayfun(@(i) hamming74_decode(rx(i:i+6)), 1:7:
    numel(rx), 'UniformOutput', false));
        case 'rep5',      out = rep_decode(rx);
    end
    out = out(1:numel(src));
    ber = mean(out ~= src);
    try
        back = prefix_decode(char(out + '0'), sym, hf);
    catch
        back = '';                                % broken tail after errors
    end
    fprintf('STEP 9-12 %-9s sent %5d bits, residual BER %.4f, text recovered exactly: %d\n'
    , ...
            schemes{s}, numel(tx), ber, strcmp(back, text));
end
```

OUTPUT (real run)

```
STEP 1-3  520 symbols, 24 distinct, H = 4.0957 bits/symbol
STEP 4-8  SF 2144+592, Huffman 2144+592, arithmetic 2131+592, LZW 2276 (raw 4160)
STEP 9-12 none      sent  2144 bits, residual BER 0.0075, text recovered exactly: 0
STEP 9-12 hamming74 sent  3752 bits, residual BER 0.0000, text recovered exactly: 1
STEP 9-12 rep5      sent 10720 bits, residual BER 0.0000, text recovered exactly: 1
```

B.13 Tests against the Python library

test_itlab_matlab.m

```
%TEST_ITLAB_MATLAB  Tests of the MATLAB labs, including comparison with
%   reference results of the Python library (pyref_it.mat).
R = load('pyref_it.mat');
syms = 'ABCDEFGHIJ';
cellrow = @(x) strsplit(strtrim(x), ' ');
okSF = 0; okHF = 0;
for t = 1:50
    k = R.K(t); p = R.P(t, 1:k);
    sf = shannon_fano_code(syms(1:k), p);  hf = huffman_code(syms(1:k), p);
    okSF = okSF + isequal(sf, cellrow(R.SF{t}));
    okHF = okHF + isequal(hf, cellrow(R.HF{t}));
end
fprintf('Shannon-Fano codes identical to Python: %d / 50\n', okSF);
fprintf('Huffman codes identical to Python     : %d / 50\n', okHF);
okA = 0; okL = 0; okRT = 0;
for t = 1:30
    txt = R.T{t};
    [sym, ~, idx] = unique(txt); counts = accumarray(idx(:), 1)';
    b = arith_encode(txt, sym, counts);
    okA = okA + strcmp(b, R.AB{t});
    okRT = okRT + strcmp(arith_decode(b, sym, counts, numel(txt)), txt);
    c = lzw_encode(txt, 'ABCDE');
    okL = okL + isequal(c, str2num(R.LZ{t})) * strcmp(lzw_decode(c, 'ABCDE'), txt); %#ok<
    ST2NM>
end
fprintf('arithmetic bit streams identical      : %d / 30 (round trips %d / 30)\n', okA,
    okRT);
fprintf('LZW codes identical and round trip    : %d / 30\n', okL);
okB = 0; okE = 0;
for t = 1:20
    c = bch1517_encode(R.M(t, :));
    okB = okB + isequal(c, R.C(t, :));
    for i = 0:14
        for j = i:14
            r = c; r(i+1) = 1 - r(i+1);
            if j > i, r(j+1) = 1 - r(j+1); end
            [~, ch] = bch1517_decode(r);
            okE = okE + isequal(ch, c);
        end
    end
end
fprintf('BCH codewords identical               : %d / 20\n', okB);
fprintf('BCH 1- and 2-error patterns corrected : %d / %d\n', okE, 20 * 120);
okH = 0;
for v = 0:15
    d = bitget(v, 4:-1:1); c = hamming74_encode(d);
    for e = 1:7
        r = c; r(e) = 1 - r(e);
        okH = okH + isequal(hamming74_decode(r), d);
    end
```

```
end
fprintf('Hamming single errors corrected       : %d / 112\n', okH);
okR = 0;
for v = 0:31
    e = bitget(v, 1:5);
    okR = okR + (isequal(rep_decode(e), 0) == (sum(e) <= 2));
end
fprintf('(5,1) patterns behaving as predicted  : %d / 32\n', okR);
```

OUTPUT (real run)

```
Shannon-Fano codes identical to Python: 50 / 50
Huffman codes identical to Python     : 50 / 50
arithmetic bit streams identical      : 30 / 30 (round trips 30 / 30)
LZW codes identical and round trip    : 30 / 30
BCH codewords identical               : 20 / 20
BCH 1- and 2-error patterns corrected : 2400 / 2400
Hamming single errors corrected       : 112 / 112
(5,1) patterns behaving as predicted  : 32 / 32
```

`pyref_it.mat` contains codes, bit streams and codewords produced by the Python library for random inputs. The MATLAB functions reproduce all of them exactly, and correct all 2400 one- and two-error patterns of BCH(15,7).